Sistemas Embebidos — Trabajo Práctico N° 1

Startup, Linker y arranque real en STM32F103

Alumno: Máximo Daniel Mariani

Legajo: 1151848

Docente: Dr. Ing. Leonardo J. Amet

Fecha de entrega: 22 de abril de 2026

1. Objetivo

El objetivo de este TP es entender, de manera concreta y verificable, cómo un STM32F103 arranca y llega a ejecutar main(). Más allá de lograr que un programa compile y se grabe, se busca poder identificar qué resuelve el código de startup y qué resuelve el linker script, cómo se distribuye la memoria entre Flash y RAM, y qué evidencia concreta demuestra que el arranque es correcto. Para forzar ese nivel de comprensión, el práctico propone inducir tres fallas, predecir sus síntomas, observarlos en placa y corregirlos con evidencia de causa raíz.

2. Estructura del proyecto

El trabajo se realizó sobre el repositorio bluepill-baremetal provisto por la cátedra. Los cuatro archivos centrales son: src/main.c (aplicación de blink sobre PC13; se le agregaron dos variables testigo: g_blink_count = 3 en .data y g_zero_marker sin inicializar en .bss); src/startup.c (vector table, handlers como alias débiles de Default_Handler, y Reset_Handler que copia .data de Flash a RAM, limpia .bss y llama a main); linker.ld (regiones FLASH 64 KB en 0x08000000 y RAM 20 KB en 0x20000000, ubicación de secciones, y símbolos __reset_stack_pointer, _sdata, _edata, _sbss, _ebss, _load_address); y Makefile (build con arm-none-eabi-gcc, flasheo con OpenOCD).

Elemento	Archivo / símbolo / sección	Función
Reset Handler	startup.c, función Reset_Handler(); dirección en vector_table[1]	Copia .data de Flash a RAM, pone .bss en cero, llama a main().
Stack inicial	__reset_stack_pointer = ORIGIN(RAM)+LENGTH(RAM) = 0x20005000	Valor cargado automáticamente en MSP al reset, leyendo *(0x08000000). Tope de la RAM.
.isr_vector	startup.c con section(".isr_vector"). En linker.ld: KEEP(*(.isr_vector)) al inicio de FLASH	Array de 60 punteros: [0] SP inicial, [1] Reset_Handler, siguientes: handlers de excepciones e IRQs.
.data	.data : { _sdata = .; *(.data*); _edata = .; } > RAM AT > FLASH. LMA = _load_address, VMA en RAM	Globales inicializadas. Única sección con VMA ≠ LMA: el startup copia de Flash a RAM.
.bss	.bss : { _sbss = .; *(.bss*); _ebss = .; } > RAM	Globales no inicializadas. El startup las pone en cero. No ocupan Flash.

2.1 Preguntas de comprensión

1. ¿Por qué main no puede ejecutarse correctamente si antes no se inicializa .data?

Porque las variables globales que uno declara con un valor inicial, tipo int x = 42, en realidad no arrancan con ese valor solas. El 42 está guardado en Flash como un dato estático, pero cuando el micro se enciende, la RAM tiene basura. Si el Reset_Handler no copia ese valor de Flash a RAM antes de llamar a main(), la variable arranca con lo que haya quedado en esa dirección de RAM. El programa corre, pero con datos equivocados desde el primer instante.

2. ¿Qué diferencia hay entre una variable global inicializada y una no inicializada durante el arranque?

Una inicializada (int x = 42) vive en .data. Tiene dos direcciones: una en Flash donde está guardado el 42, y otra en RAM donde realmente va a estar cuando el programa corra. El startup la copia de Flash a RAM antes de main(). Una no inicializada (int y) vive en .bss. No hay nada guardado en Flash (sería gastar Flash en guardar ceros), pero el estándar C dice que tiene que arrancar en cero, así que el startup se encarga de escribir ceros en su lugar en RAM. La primera consume Flash y RAM; la segunda solo RAM.

3. ¿Qué parte del proyecto define dónde vive cada sección de memoria?

El linker script, linker.ld. Ahí están declaradas las regiones de memoria del chip (la Flash de 64 KB en 0x08000000 y la RAM de 20 KB en 0x20000000), y en el bloque SECTIONS se le dice a cada parte del binario a qué región va: .text a Flash, .bss a RAM, .data el caso especial con > RAM AT > FLASH. El linker también genera los símbolos (_sdata, _edata, _sbss, _ebss, _load_address, __reset_stack_pointer) que después usa el startup para saber qué copiar y hasta dónde.

4. ¿Qué parte del proyecto ejecuta la copia o limpieza de memoria?

El archivo src/startup.c, más específicamente la función Reset_Handler. Usa los símbolos que el linker dejó listos y hace dos cosas en dos loops: primero copia word a word desde _load_address en Flash hasta el rango _sdata..._edata en RAM (esto es .data), y después recorre de _sbss a _ebss poniendo ceros (esto es .bss). Recién cuando termina estas dos cosas llama a main().

3. Descripción del arranque

La secuencia tiene cinco pasos. Los dos primeros son obligatorios por hardware y no pueden intervenirse: (1) el CPU lee el word en 0x08000000 y lo carga en el MSP — en este proyecto 0x20005000; (2) lee el word en 0x08000004 y salta a esa dirección, que es el Reset_Handler. (3) El Reset_Handler recorre desde _sdata hasta _edata copiando desde _load_address en Flash: así se materializan los valores iniciales de .data. (4) Recorre desde _sbss hasta _ebss escribiendo ceros. (5) Llama a main().

La distinción VMA/LMA es central. Para .text y .bss ambas coinciden. Para .data, VMA está en RAM (donde la variable vive en ejecución) y LMA en Flash (donde está guardado el valor inicial). El puente entre las dos regiones lo hace el Reset_Handler durante el arranque.

4. Evidencia de versión sana

Toda la evidencia está en evidencia/ ubicada en el anexo. Los artefactos más relevantes:

Tamaño de secciones (size)

```
   text    data     bss     dec     hex  filename
    512       4       4     520     208  bin/blink_minimal.elf
```

Los 4 bytes de .data son g_blink_count; los 4 de .bss, g_zero_marker. Uso total: 520 bytes sobre 65.536 de Flash.

Símbolos clave (nm)

```
08000000 R vector_table          20000000 B g_zero_marker
080000d4 T main                  20000004 D g_blink_count
080001a4 T Reset_Handler         20000008 D _edata
08000200 A _load_address         20005000 R __reset_stack_pointer
```

LMA vs VMA (objdump -h) — la prueba documental clave

```
Idx  Name      Size      VMA       LMA
  0  .vectors  000000d4  08000000  08000000
  1  .text     0000012c  080000d4  080000d4
  2  .bss      00000004  20000000  20000000
  3  .data     00000004  20000004  08000200
```

La sección .data aparece con VMA = 0x20000004 (RAM) y LMA = 0x08000200 (Flash). Es la única con VMA ≠ LMA, como dicta la directiva > RAM AT > FLASH.

Estado del CPU (OpenOCD)

```
xPSR: 0x01000000   pc: 0x080001a4   msp: 0x20005000
```

Los tres registros confirman arranque sano: bit T = 1 (modo Thumb), pc coincide con Reset_Handler en nm, msp coincide con __reset_stack_pointer. El log del hardware cierra el círculo con lo que calculó el linker y reportó el ELF.

En placa, el LED de PC13 parpadea 3 veces, pausa larga, y repite. Esto prueba que .data se copió bien (g_blink_count = 3) y .bss se limpió (g_zero_marker = 0 hace que el if se cumpla).

4.1 Respuestas a preguntas obligatorias de la Etapa 2

1. ¿Desde qué dirección toma el micro el stack inicial al reset?

Cuando el micro se resetea, lo primero que hace el Cortex-M3 es leer los 4 bytes que están en la dirección 0x08000000 y copiarlos al MSP (el stack pointer). Eso está hecho así por hardware, no se puede cambiar. En este proyecto, ese valor es __reset_stack_pointer = 0x20005000, que es el tope de la RAM. Lo confirman dos cosas: nm me lo muestra en esa dirección, y el log de OpenOCD después del reset dice msp: 0x20005000. Coinciden exactamente.

2. ¿Dónde está ubicada la vector table del proyecto?

En 0x08000000, al principio de la Flash. Es obligatorio que esté ahí, porque es la dirección que el hardware va a leer cuando se resetee. En el linker script, la sección .vectors es la primera declarada dentro de SECTIONS y va a FLASH, así que queda alineada con ORIGIN(FLASH) = 0x08000000. nm lo confirma (08000000 R vector_table) y objdump -h también, con tamaño 0xd4 bytes (son 53 entradas de 4 bytes: el SP inicial, el Reset_Handler, y 51 handlers más). Además, el linker la protege con KEEP(...) para que no la descarte si le parece que no se usa.

3. ¿Qué evidencia muestra que .data se ejecuta en RAM aunque su contenido inicial se almacena en Flash?

La prueba más clara está en objdump -h: la sección .data aparece con VMA 0x20000004 (RAM) y LMA 0x08000200 (Flash). Son direcciones distintas, y eso es único de .data: en todas las otras secciones VMA y LMA coinciden. Esto pasa por la directiva > RAM AT > FLASH del linker, que le dice "esta sección va a ejecutarse en RAM pero los valores iniciales cárgalos en Flash". El símbolo _load_address captura la LMA y el Reset_Handler la usa para saber desde dónde copiar al arranque.

4. ¿Qué evidencia muestra que .bss arranca en cero?

Mirando de nuevo objdump -h, la sección .bss aparece con el atributo ALLOC pero no tiene CONTENTS, LOAD, a diferencia de .data que sí los tiene. Eso quiere decir que en el archivo ELF no hay bytes guardados para .bss: el espacio se reserva pero el contenido no viene en el binario. Es el Reset_Handler el que se encarga de recorrer desde _sbss hasta _ebss escribiendo ceros antes de llamar a main(). La prueba funcional está en la placa: g_zero_marker vive en .bss, y en main() el if (g_zero_marker == 0) se cumple y se ejecuta el blink de 3 pulsos. Si .bss no se hubiera limpiado, habría entrado al else y el LED habría quedado fijo.

5. Fallas inducidas y correcciones

Cada falla se indujo sobre copia sana del proyecto, con predicción escrita antes de ejecutar, y se restauró para demostrar recuperación. Evidencia completa en evidencia/falla_A/, falla_B/ y falla_C/.

5.1 Falla A — Vector table mal ubicada

Intervención: en el bloque SECTIONS de linker.ld se declaró .text antes de .vectors. El linker coloca las secciones en el orden declarado, así que .vectors queda desplazada del inicio de Flash.

Predicción: al reset, el CPU lee incondicionalmente 0x08000000 y 0x08000004 para MSP y Reset_Handler. Con la intervención, esas direcciones contienen las primeras instrucciones Thumb del código, no la vector table. Se espera HardFault inmediato: el LED ni siquiera llega a parpadear.

Evidencia del binario defectuoso (objdump -h):

```
Idx  Name      Size      VMA       LMA
  0  .text     0000012c  08000000  08000000  ← usurpó el lugar
  1  .vectors  000000d4  0800012c  0800012c  ← desplazada
```

Los primeros 16 bytes en 0x08000000 son el prólogo Thumb de main() (80b4 85b0 00af 294b). Interpretados como datos por el hardware: MSP = 0xb08580b4 (fuera de RAM), handler = 0x4b29af00 (fuera de Flash).

Evidencia del hardware (OpenOCD capturó el CPU en HardFault):

```
xPSR: 0x00000000   pc: 0x4b29af00   msp: 0xb085b480
```

Los tres registros confirman el fallo simultáneamente: xPSR con bit T = 0 (imposible en Cortex-M3, característico de HardFault), pc fuera de Flash, msp fuera de RAM. Todo roto.

Síntoma en placa: LED apagado. El CPU no ejecuta una sola línea del código del usuario.

Corrección: se restauró el orden original de las secciones en linker.ld. El diff con el respaldo quedó vacío, objdump -h confirma .vectors de vuelta en 0x08000000, y el blink reapareció.

5.2 Falla B — Inicialización de .data incorrecta

Intervención: se reemplazó _load_address = LOADADDR(.data) por _load_address = 0x08000000. El símbolo que el startup usa como origen para copiar .data apunta ahora al inicio de Flash (vector table) en vez de a 0x08000200 (LMA real).

Predicción: el Reset_Handler ejecutará su bucle normalmente pero leerá desde 0x08000000, donde están los 4 bytes de __reset_stack_pointer (0x20005000). Por lo tanto g_blink_count valdrá 0x20005000 = 536.891.392 en vez de 3. El blink ejecutará esa cantidad enorme de iteraciones por serie, haciendo que la pausa larga nunca aparezca. No habrá HardFault: el programa corre con datos envenenados.

Evidencia del binario defectuoso (nm):

```
Sana:       08000200 A _load_address
Con falla:  08000000 A _load_address
```

Detalle importante: el objdump -h sigue mostrando LMA = 0x08000200 para .data, porque el linker siguió poniendo los valores ahí. El bug no es que falten los datos: es que el startup los busca en otro lado. Comparando bytes físicos: en 0x08000200 hay 03 00 00 00 (el valor correcto); en 0x08000000 hay 00 50 00 20 (el stack pointer). El startup copia este último.

Síntoma en placa: LED parpadeó al ritmo de los delays internos (200.000 ciclos ON/OFF) sin pausas largas visibles. El bucle for (k = 0; k < g_blink_count; k++) pasó a tener 500 millones de iteraciones y la pausa larga habría aparecido recién después de días.

Corrección: se restauró _load_address = LOADADDR(.data). El símbolo volvió a 0x08000200 y el blink de 3 pulsos reapareció.

5.3 Falla C — Margen de stack insuficiente

Intervención: __reset_stack_pointer = ORIGIN(RAM) + LENGTH(RAM) fue reemplazado por ORIGIN(RAM) + 16. El stack arranca en 0x20000010, dentro del rango de las variables globales (.bss en 0x20000000, .data en 0x20000004).

Predicción: el stack crece hacia abajo, así que el primer push del prólogo de main() sobreescribe .bss y .data con registros guardados. g_zero_marker y g_blink_count quedan con valores basura. A diferencia de la Falla B, el daño es continuo: cada push posterior reescribe esas direcciones. Por eso se esperaba comportamiento más errático que en la Falla B.

Evidencia del binario defectuoso (nm):

```
20000010 R __reset_stack_pointer
```

Los primeros 4 bytes del binario (objdump -s -j .vectors) son 10 00 00 20 → 0x20000010. Ese valor queda grabado en la Flash y es lo que el CPU cargará en MSP al reset.

Evidencia del hardware (OpenOCD capturó el CPU ejecutando la Falla C):

```
xPSR: 0x01000000   pc: 0x080001a4   msp: 0x20000010
```

Estado conceptualmente distinto al de la Falla A. Dos de tres registros son normales: xPSR con bit T = 1 (Thumb válido), pc = 0x080001a4 (Reset_Handler exactamente). El único anómalo es el msp = 0x20000010, coincidente con el __reset_stack_pointer defectuoso. No es HardFault: es un arranque sintácticamente válido con un contrato de memoria roto.

Síntoma en placa: LED encendido fijo en todos los resets probados (más de 10). No hubo variabilidad, a pesar de que la predicción la contemplaba. Se debe a que este programa no tiene fuentes de no-determinismo (sin interrupciones, sin entradas externas, mismo prólogo de main() compilado siempre igual): los pushes caen en las mismas direcciones con los mismos valores, corrompen g_zero_marker siempre con el mismo valor ≠ 0, y el código cae en la rama else que deja el LED fijo.

Por qué esta falla tiene más potencial errático que la B: en la Falla B la causa raíz es una copia única en un momento fijo; el resultado es determinista por construcción. En la Falla C el daño es continuo: cada llamada a función, cada push, cada interrupción mueve el SP y reescribe zonas distintas. En este programa minimalista el determinismo emergió por falta de variabilidad, pero en un proyecto con interrupciones o entradas asíncronas el mismo cambio daría comportamiento diferente entre ejecuciones.

Corrección: __reset_stack_pointer volvió a ORIGIN(RAM) + LENGTH(RAM) = 0x20005000. El stack quedó lejos de las variables globales y el blink de 3 pulsos reapareció estable.

6. Conclusión técnica

Antes de hacer este TP, si veía un proyecto embebido lo único que miraba era main.c. Me parecía que ahí estaba todo: las variables, la configuración de los periféricos, el loop principal. El arranque del micro me resultaba algo que simplemente pasaba, como si el programa empezara en main() porque sí.

Lo que me quedó claro después del TP es que entre el reset y la primera línea de main() pasan un montón de cosas, y todas tienen que salir bien para que el código C haga lo que parece que va a hacer. El hardware del Cortex-M3 obliga a que en la dirección 0x08000000 haya un valor válido de stack pointer, y en 0x08000004 la dirección del Reset_Handler. Si esas dos cosas no están donde corresponden, el micro ni siquiera llega a ejecutar una sola línea del código que escribí yo. Eso fue lo que me pasó en la Falla A.

También entendí que cuando declaro algo como uint32_t g_blink_count = 3, ese 3 no está ya en RAM cuando arranca el micro. Los 4 bytes están guardados en Flash, y es el Reset_Handler el que los copia a RAM antes de llamar a main(). Si ese bucle de copia lee desde la dirección equivocada, la variable arranca con otro valor y main() se ejecuta normalmente pero haciendo cosas que no tienen sentido. En la Falla B observamos exactamente eso: el programa andaba, el LED parpadeaba, pero lo hacía con un ritmo completamente distinto porque la variable valía 0x20005000 en vez de 3.

La Falla C fue la más interesante porque me mostró un tipo de problema distinto: el micro arrancaba perfecto, el Reset_Handler se ejecutaba, main() se llamaba, pero como el stack arrancaba dentro del rango de las variables globales, el primer push del prólogo las pisaba. El código corría pero leyendo datos que él mismo acababa de sobreescribir. Lo más curioso fue que el síntoma fue idéntico en todos los resets, a pesar de que la predicción decía que podía ser errático. Terminé entendiendo que el carácter errático aparece cuando hay interrupciones u otras fuentes de variabilidad; en un programa sencillo como este, el bug es perfectamente reproducible.

Otra cosa que no tenía clara antes es que el linker no te avisa si estás rompiendo algo. Podés escribir un linker.ld que mande todo a donde no corresponde, el toolchain compila, objcopy genera el .bin, OpenOCD dice Verified OK, y el firmware no anda. La ausencia de errores en la compilación no significa nada; lo que importa es que los símbolos que usa el startup apunten donde corresponde y que el mapa de memoria respete las reglas del hardware. Para eso hay que mirar objdump -h, nm y el estado del CPU tras el reset. Esos son los que te dicen la verdad.

En síntesis: main.c es apenas una parte del proyecto. Para que arranque y funcione bien, el startup.c y el linker.ld tienen que estar bien coordinados con el hardware. Si algo falla ahí, main() nunca va a comportarse como uno espera, por más correcto que sea su código.

Anexo — Organización de la evidencia

Toda la evidencia técnica referida en el informe está en el repositorio adjunto en el anexo bajo evidencia/, organizada por etapa: evidencia/ contiene salidas de size, nm, objdump, recortes del .map, log de OpenOCD y video GIF del blink funcionando. evidencia/falla_A/, falla_B/ y falla_C/ contienen cada una: predicción previa a ejecución, diff del cambio en linker.ld, evidencia binaria del defecto, salida del CPU tras reset, y artefactos de la corrección. Cada carpeta incluye el .map completo correspondiente, permitiendo reconstruir cualquier análisis sin volver a compilar.

Repositorio en GitHub: maxmariani01/TP1-Embebidos-Mariani